

Token Management System

Objectives

- To provide an organized and automated queue management system for hospitals and healthcare facilities.
- To eliminate manual queue handling and reduce patient waiting confusion.
- To enable real-time token generation using an IoT-based kiosk.
- To provide doctors with a centralized dashboard for efficient patient queue management.
- To display the currently serving token and waiting queue in real time.
- To demonstrate seamless integration between ESP32, Flask, SQLite, and a live web dashboard.

System Overview

The Token Management System is designed to provide an organized and efficient patient queue management solution for hospitals and healthcare facilities. It enables patients to generate queue tokens electronically while allowing doctors to manage the consultation queue through a dedicated dashboard. A public display keeps waiting patients informed of the currently served token and the remaining queue.

The system consists of three main components:

1. Token Generation Kiosk

Installed at the hospital reception or OPD entrance, this device allows patients to generate a queue token.

The kiosk consists of:

- ESP32 Development Board
- Capacitive Touch Sensor (or Push Button)
- Wireless Wi-Fi Connectivity
- Token Generation Interface

Whenever a patient touches the sensor, a request is sent to the server, which generates the next available token number.

2. Doctor Dashboard

Located inside the doctor's consultation room, this interface allows the doctor to manage the patient queue.

The dashboard displays:

- Currently Serving Token
- Number of Waiting Patients
- Next Token Button
- Queue Status

The doctor advances the queue by pressing the **Next Token** button after completing the consultation or treatment of the current patient.

3. Public Display Interface

Installed in the patient waiting area.

The display shows:

- Currently Serving Token
- Waiting Queue
- Live Queue Updates

The display updates automatically whenever a new token is generated or the doctor advances the queue.

Working Principle

When a patient arrives at the hospital, they generate a token by touching the capacitive touch sensor at the token generation kiosk.

The ESP32 sends a request to the central Flask server over Wi-Fi.

The server generates the next available token (A001, A002, A003...) and stores it in the SQLite database with the status **Waiting**.

The newly generated token immediately appears on the kiosk as the latest issued token.

Inside the consultation room, the doctor uses the Doctor Dashboard to call the next patient by pressing the **Next Token** button.

The server changes the status of the previously serving token to **Completed** and promotes the oldest waiting token to **Serving**.

The Public Display Interface updates automatically to show the currently serving token and the remaining waiting queue.

The process continues until all waiting patients have been attended.

System Workflow

Patient Arrives



Touches Token Generation Sensor



ESP32 Sends Request to Server



Server Generates New Token



Database Stores Token (Waiting)



Doctor Presses **Next Token**



Current Serving Token → Completed



Next Waiting Token → Serving



Public Display Updates Automatically



Patient Proceeds for Consultation



Queue Continues

Queue States

Each generated token progresses through the following states:

Waiting

The patient has generated a token and is waiting to be called.

Serving

The patient has been called by the doctor and is currently being consulted.

Completed

The patient's consultation has been completed, and the token is removed from the active queue.

Requirements Summary

Requirement	Description
Token Generation	Patients should be able to generate a token using a capacitive touch sensor or push button.
Automatic Numbering	The server should generate sequential token numbers without duplication.
Wireless Communication	The ESP32 should communicate reliably with the Flask server over Wi-Fi.
Queue Storage	All generated tokens should be stored in a SQLite database.
Queue Management	The doctor should be able to call the next waiting patient using the Doctor Dashboard.
Public Display	The currently serving token and waiting queue should be displayed in real time.
Status Management	Every token should transition through the states Waiting → Serving → Completed.
Real-Time Synchronization	All connected interfaces should receive queue updates instantly using Socket.IO without requiring manual refresh.

Step-by-Step Sequence with Requirements

Step 1: Patient Generates Token

- The patient touches the capacitive touch sensor at the Token Generation Kiosk.
- The ESP32 sends a request to the Flask server.

Requirements

- Reliable single-touch registration.
- Immediate wireless communication.
- Fast server response.
- Prevention of duplicate token generation.

Step 2: Token Generation

- The server generates the next sequential token.
- The token is stored in the database with the status **Waiting**.
- The kiosk displays the latest generated token.

Requirements

- Sequential token numbering.
- No duplicate tokens.
- Automatic database update.
- Immediate confirmation of token generation.

Step 3: Doctor Calls Next Patient

- The doctor presses the **Next Token** button on the Doctor Dashboard.
- The current serving token is marked as **Completed**.
- The oldest waiting token is promoted to **Serving**.

Requirements

- FIFO (First In First Out) queue management.
- Automatic status transition.
- No skipped or duplicated tokens.
- Instant synchronization with all connected interfaces.

Step 4: Public Display Updates

- The server broadcasts the updated queue information.
- The Public Display Interface immediately updates the currently serving token and waiting queue.

Requirements

- Real-time updates.
- No manual page refresh.
- Large, clear, and easily readable interface.
- Automatic synchronization with the server.

Technology Stack

Component	Technology
Microcontroller	ESP32 DevKit V1
Input Device	Capacitive Touch Sensor (ESP32 Touch Pin)
Backend Server	Flask (Python)
Database	SQLite
Real-Time Communication	Flask-SocketIO
Frontend	HTML, CSS, JavaScript
Communication Protocol	HTTP over Wi-Fi
Development Environment	Arduino IDE, Visual Studio Code

Future Enhancements

- Multiple Doctor Dashboards
- Multiple Consultation Rooms

- Priority Tokens for Emergency Cases
- Voice Announcement System ("Token A025, please proceed to Consultation Room 1.")
- Thermal Receipt Printer for Token Printing
- SMS/WhatsApp Token Notifications
- Estimated Waiting Time Display
- Queue Recall Function
- End-of-Day Queue Reset
- Patient Appointment Integration
- Daily Consultation Analytics
- Hospital Information System (HIS) Integration

